

Factory Method Design Pattern

Exercise 2 — Complete Answer with Java Source Code

Project: `FactoryMethodPatternExample`

Topic: Creational Design Patterns | Java

Pattern Type	Creational
Intent	Define interface for creating an object; subclasses decide which class to instantiate
Key Participants	Document (interface), Concrete Documents, DocumentFactory (abstract), Concrete Factories
Benefit	Decouples object creation from usage — add new types without changing existing code

STEP 1

Project Setup — FactoryMethodPatternExample

Create a new Java project named **FactoryMethodPatternExample**. The project contains the following source files, all inside the **src/** folder:

```
FactoryMethodPatternExample/  
  
src/  
  
    Document.java           (interface)  
    WordDocument.java       (concrete class)  
    PdfDocument.java        (concrete class)  
    ExcelDocument.java      (concrete class)  
    DocumentFactory.java    (abstract factory)  
    WordDocumentFactory.java (concrete factory)  
    PdfDocumentFactory.java (concrete factory)  
    ExcelDocumentFactory.java (concrete factory)  
    FactoryMethodTest.java  (test / driver)
```

STEP 2

Define the Document Interface

All document types share a common contract defined by the **Document** interface. This allows the factory and client code to work with any document type without knowing its concrete class — the essence of the pattern.

```
// File: Document.java  
  
// File: Document.java  
  
public interface Document {  
  
    // Every document must support these three operations  
    void open();  
    void save();  
    void close();  
  
    // Returns the document type name (useful for logging / display)  
    String getDocumentType();  
}
```

◆ Using an interface (rather than an abstract class) gives maximum flexibility — concrete classes can still extend another class if needed.

STEP 3

Implement Concrete Document Classes

Each concrete class implements the **Document** interface and provides its own specific behaviour for open, save, and close operations.

WordDocument.java

```
// File: WordDocument.java

public class WordDocument implements Document {

    private String fileName;

    public WordDocument(String fileName) {
        this.fileName = fileName;
    }

    @Override
    public void open() {
        System.out.println("[Word] Opening document: " + fileName + ".docx");
        System.out.println("[Word] Loading Microsoft Word format...");
    }

    @Override
    public void save() {
        System.out.println("[Word] Saving document: " + fileName + ".docx");
        System.out.println("[Word] Applying .docx format with styles and headers.");
    }

    @Override
    public void close() {
        System.out.println("[Word] Closing document: " + fileName + ".docx");
    }

    @Override
    public String getDocumentType() {
        return "Microsoft Word Document (.docx)";
    }
}
```

PdfDocument.java

```
// File: PdfDocument.java

public class PdfDocument implements Document {

    private String fileName;

    public PdfDocument(String fileName) {
        this.fileName = fileName;
    }

    @Override
    public void open() {
        System.out.println("[PDF] Opening document: " + fileName + ".pdf");
        System.out.println("[PDF] Rendering pages with PDF engine...");
    }
}
```

```

}

@Override
public void save() {
    System.out.println("[PDF] Saving document: " + fileName + ".pdf");
    System.out.println("[PDF] Compressing and embedding fonts.");
}

@Override
public void close() {
    System.out.println("[PDF] Closing document: " + fileName + ".pdf");
}

@Override
public String getDocumentType() {
    return "Portable Document Format (.pdf)";
}
}

```

ExcelDocument.java

```

// File: ExcelDocument.java

public class ExcelDocument implements Document {

    private String fileName;

    public ExcelDocument(String fileName) {
        this.fileName = fileName;
    }

    @Override
    public void open() {
        System.out.println("[Excel] Opening document: " + fileName + ".xlsx");
        System.out.println("[Excel] Loading spreadsheet grid and formulas...");
    }

    @Override
    public void save() {
        System.out.println("[Excel] Saving document: " + fileName + ".xlsx");
        System.out.println("[Excel] Recalculating all cell formulas before save.");
    }

    @Override
    public void close() {
        System.out.println("[Excel] Closing document: " + fileName + ".xlsx");
    }

    @Override
    public String getDocumentType() {
        return "Microsoft Excel Spreadsheet (.xlsx)";
    }
}

```

STEP 4

Implement the Factory Method

The **DocumentFactory** abstract class declares the factory method **createDocument()**. Each concrete factory subclass overrides this method to instantiate its own document type. The abstract class also provides a template method **newDocument()** that calls the factory method and then performs common steps.

DocumentFactory.java — Abstract Factory

[illegible]

WordDocumentFactory.java

```
// File: WordDocumentFactory.java

public class WordDocumentFactory extends DocumentFactory {

@Override

public Document createDocument(String fileName) {
// Decide WHICH class to instantiate – this is the factory method
return new WordDocument(fileName);
}

}
```

PdfDocumentFactory.java

```
// File: PdfDocumentFactory.java

public class PdfDocumentFactory extends DocumentFactory {

    @Override
    public Document createDocument(String fileName) {
        return new PdfDocument(fileName);
    }
}
```

ExcelDocumentFactory.java

```
// File: ExcelDocumentFactory.java

public class ExcelDocumentFactory extends DocumentFactory {
```

```
@Override
public Document createDocument(String fileName) {
    return new ExcelDocument(fileName);
}
}
```

◆ *Each concrete factory contains ONLY one line of logic — the new keyword. All orchestration (open, save, close) stays in the abstract class or client, keeping factory classes clean and easy to extend.*

STEP 5

Test the Factory Method Implementation

The test class uses each concrete factory to produce documents. Notice that the client code **only talks to DocumentFactory and Document** — it never calls **new WordDocument()** directly, achieving full decoupling.

FactoryMethodTest.java

[illegible]

```

Document doc = factories[i].newDocument(fileNames[i]);
doc.save();
doc.close();
System.out.println();
}

System.out.println("=====");
System.out.println(" All documents processed successfully.");
System.out.println("=====");
}
}

```

Expected Console Output

```

=====
Document Management System
Factory Method Pattern Demo
=====

>> Created: Microsoft Word Document (.docx)
[Word] Opening document: AnnualReport.docx
[Word] Loading Microsoft Word format...
[Word] Saving document: AnnualReport.docx
[Word] Applying .docx format with styles and headers.
[Word] Closing document: AnnualReport.docx

>> Created: Portable Document Format (.pdf)
[PDF] Opening document: UserManual.pdf
[PDF] Rendering pages with PDF engine...
[PDF] Saving document: UserManual.pdf
[PDF] Compressing and embedding fonts.
[PDF] Closing document: UserManual.pdf

>> Created: Microsoft Excel Spreadsheet (.xlsx)
[Excel] Opening document: SalesData_Q3.xlsx
[Excel] Loading spreadsheet grid and formulas...
[Excel] Saving document: SalesData_Q3.xlsx
[Excel] Recalculating all cell formulas before save.
[Excel] Closing document: SalesData_Q3.xlsx

-----
Batch Processing – Polymorphic Factory
-----

>> Created: Microsoft Word Document (.docx)
[Word] Opening document: Contract.docx
...
>> Created: Portable Document Format (.pdf)
[PDF] Opening document: Invoice.pdf
...
>> Created: Microsoft Excel Spreadsheet (.xlsx)
[Excel] Opening document: Budget.xlsx
...

=====

```


All documents processed successfully.

=====

KEY CONCEPTS

Roles in the Factory Method Pattern

Role	Class in This Project	Responsibility
Product (Interface)	Document	Declares the interface all products must follow
Concrete Product	WordDocument PdfDocument ExcelDocument	Implements the Document interface with type-specific behaviour
Creator (Abstract)	DocumentFactory	Declares the factory method createDocument(); may include default behaviour
Concrete Creator	WordDocumentFactory PdfDocumentFactory ExcelDocumentFactory	Overrides createDocument() to return its specific product
Client	FactoryMethodTest	Works only with DocumentFactory and Document; never uses 'new' directly

UML Class Diagram

```
<<interface>>
+-----+
| Document |
+-----+
| + open() : void |
| + save() : void |
| + close() : void |
| + getDocumentType():String|
+-----+
^ ^ ^
| | |
+-----+ +-----+ +-----+
|WordDocument|PdfDocument|ExcelDocument|
+-----+ +-----+ +-----+

<<abstract>>
+-----+
| DocumentFactory |
+-----+
| + createDocument(fileName) | <-- Factory Method (abstract)
| + newDocument(fileName) | <-- Template Method
+-----+
^ ^ ^
| | |
+-----+ +-----+ +-----+
|WordDocument | |PdfDocument | |ExcelDocument |
|Factory | |Factory | |Factory |
+-----+ +-----+ +-----+
|createDocument | |createDocument|createDocument |
|() : Word... | |() : Pdf... | |() : Excel... |
+-----+ +-----+ +-----+
| | |
v v v
WordDocument PdfDocument ExcelDocument
```

When to Use the Factory Method Pattern

- You don't know ahead of time which class to instantiate.
- You want subclasses to control which objects get created.
- You want to encapsulate object creation logic in one place.
- You need to add new types (e.g., MarkdownDocument) without modifying existing code (Open/Closed Principle).

Extending the System — Adding a New Document Type

One of the greatest advantages of this pattern: adding a new document type requires **zero changes** to existing classes. Simply add two new files:

```
// Step A: New concrete product
public class MarkdownDocument implements Document {
    private String fileName;
    public MarkdownDocument(String fileName) { this.fileName = fileName; }

    @Override public void open() { System.out.println("[MD] Opening: " + fileName + ".md"); }
    @Override public void save() { System.out.println("[MD] Saving: " + fileName + ".md"); }
    @Override public void close() { System.out.println("[MD] Closing: " + fileName + ".md"); }
    @Override public String getDocumentType() { return "Markdown Document (.md)"; }
}

// Step B: New concrete factory
public class MarkdownDocumentFactory extends DocumentFactory {
    @Override
    public Document createDocument(String fileName) {
        return new MarkdownDocument(fileName);
    }
}

// Client code – no existing file changed
DocumentFactory mdFactory = new MarkdownDocumentFactory();
Document mdDoc = mdFactory.newDocument("README");
mdDoc.save();
mdDoc.close();
```

◆ This follows the Open/Closed Principle — the system is open for extension (new document types) but closed for modification (existing factory/product code untouched).

Summary

The **Factory Method Pattern** defines an interface (**Document**) for creating objects, but lets **subclasses** (WordDocumentFactory, PdfDocumentFactory, ExcelDocumentFactory) decide which class to instantiate.

The client (**FactoryMethodTest**) interacts only with **DocumentFactory** and **Document** — it never uses **new** directly on a concrete class, making the system highly extensible and loosely coupled.

Adding a new document type needs **only 2 new files** and **zero changes** to existing code.